Numerical Computing

Course Details

Course Code: CS-2008

Course Title: Numerical Computing

Credit Hours: 3

Instructor: Muhammad Rizwan

Grading: Absolute Grading

Prerequisite: Linear Algebra

Dr. Muhammad Rizwan

Assistant Professor

Office: **D109H**

🕛 Schedule and Office Hours (Fall-2025)						
Days	**8:30 AM 9:50 AM**	**10:00 AM 11:20 AM**	**11:30 AM 12:50 PM**	**1:00 PM 2:20 PM**	**2:20 PM 3:50 PM**	**3:55 PM 4:50 PM**
Monday	D-304 Ds-CS-D		Office Hours	Office Hours		
Tuesday	D-301 Ds-CS-E		Office Hours	Office Hours		
Wednesday	D-302 Ds-CS-D		Office Hours	Office Hours		
Thursday	C-311 Ds-CS-E		Office Hours	D-312 Numerical		
Friday			C-310 Numerical			

Profile Summary

🎓 Education:

- PhD in Computer Science, Soongsil University, Korea
- MS in Computational Sciences, NUST, Pakistan
- BS in Computer Science, FAST-NUCES
- BSc in Math & Physics, BZU Multan

💼 Work Experience:

- 14+ years in Software, IT, HPC R&D
- Roles: Sr. Program Manager, Project Manager, Deputy Manger IT, Principal Engineer
- Organizations: PITB, NimbleGeeks, PMA, Techlogix

📚 Publications:

- 6+ peer-reviewed papers (IEEE, BMC, Elsevier, Springer) Google Scholar

🧠 Patents:

- VacSol – Reverse Vaccinology Pipeline
- PanRV – Pangenome Reverse Vaccinology

📜 Certifications:

- PMP (PMI), AWS Cloud Practitioner (Professional Certifications)

Course Topics Overview

15 Weeks – 3 contact hours per week

Topics include:

- Introduction & Python for Numerical Computing
- Computer Arithmetic & Sources of Error
- Roots of Equations
- Interpolation & Approximation
- Numerical Integration
- Numerical Differentiation & ODEs
- Numerical Linear Algebra
- Project

Course Topics Overview

Textbook(s)	▶ Computational Mathematics: An introduction to Numerical Analysis and Scientific Computing with Python, Chapman and Hall/CRC, Dimitrios Mitsotakis
Reference Material	▶ Fundamentals of Numerical Computation by Toby Driscoll and Rich Braun ▶ Classical Numerical Analysis: A Comprehensive Course By Abner J. Salgado, Steven M. Wise

Topics to Cover

- Introduction & Python for Numerical Computing
- Computer Arithmetic & Sources of Error
- Roots of Equations
- Interpolation & Approximation
- Numerical Integration
- Numerical Differentiation & ODEs
- Numerical Linear Algebra

Topics to Cover (Today)

- **Introduction & Python for Numerical Computing**
- **Computer Arithmetic & Sources of Error**
  - Floating-point representation and precision issues
  - Discretization error, catastrophic cancellation
  - Scientific notation, rounding, truncation
  - Sequences, convergence, Taylor series
  - Approximations using Intermediate Value Theorem
- Roots of Equations
- Interpolation & Approximation
- Numerical Integration
- Numerical Differentiation & ODEs
- Numerical Linear Algebra
- Project

Computer Arithmetic & Sources of Error

Floating-point representation and precision issues

Learning Objectives:

- Represent a real number in a floating point system
- Compute the memory requirements of storing integers versus double precision
- Define Machine Epsilon
- Identify the smallest representable floating point number

Computer Arithmetic & Sources of Error

Represent a real number in a floating point system

To convert a **fractional decimal** (like 0.1) to binary:

1. Multiply the fraction by 2.
2. Record the **integer part** (0 or 1) — this becomes the next bit.
3. Repeat using the new **fractional part**.
4. Continue until the fraction becomes zero *or* starts repeating.

Example for 0.1:

- 0.1 × 2 = 0.2 → integer bit: 0
- 0.2 × 2 = 0.4 → bit: 0
- 0.4 × 2 = 0.8 → bit: 0
- 0.8 × 2 = 1.6 → bit: 1
- 0.6 × 2 = 1.2 → bit: 1
- 0.2 × 2 = 0.4 → bit: 0 → and now it **cycles** back!

Result:

$$0.1_{10} = 0.0001100110011\ldots_2 \quad \text{(repeating)}$$

Computer Arithmetic & Sources of Error

Represent a real number in a floating point system

- Decimal fractions that **terminate** only in binary if their denominator (in simplest form) is a power of 2.
- Example: 0.5 (which is 1/2) becomes $\mathbf{0.1_2}$, terminating nicely.
- However, 0.1 (which is 1/10) does **not terminate** in binary because:
  - Its denominator (10) includes a factor of 5, and powers of 2 don't incorporate 5.
  - Thus its binary expansion goes on repeating indefinitely.
- This is why **0.1—and many other decimal fractions—can't be represented exactly** in binary floating-point.

Computer Arithmetic & Sources of Error

Represent a real number in a floating point system

- Computers must **truncate or round** such repeating binary fractions to fit fixed-size storage (e.g., 23 or 52 bits).
- As a result:
  - **0.1 isn't stored exactly**, but as the closest representable binary number.
  - This leads to **tiny errors** that can become visible in computations like `0.1 + 0.2` not equaling exactly `0.3`.
- In essence:
  - **Not every decimal can be perfectly captured** in binary.
  - Floating-point systems introduce **inherent rounding differences**, which sometimes surface in surprising ways.

Computer Arithmetic & Sources of Error

Floating-point representation and precision issues

Fixed-Point vs Floating-Point Storage

- **Fixed-point:** Uses a fixed split of bits between integer and fraction parts.
  - Example: 64 bits total → 32 bits integer + 32 bits fraction.
- **Floating-point:** Like scientific notation in binary:
  - Stored as $\pm q \times 2^m$, where q is the **significand** and m is the exponent.
  - The significand is typically normalized: $q = 1.f$, with the leading 1 implied (not stored)—this is the *hidden-bit*

Computer Arithmetic & Sources of Error

Floating-point representation and precision issues

Fixed-Point vs Floating-Point Storage

A floating-point number can represent numbers of different orders of magnitude(very large and very small) with the same number of fixed digits.

More formally, we can define a floating point number x as:

$$x = \pm q \cdot 2^m$$

where:

- $\pm$ is the sign
- q is the significand
- m is the exponent

Aside from the special case of zero and subnormal numbers (discussed below), the significand is always in normalized form:

$$q = 1.f$$

where:

- f is the fractional part of the significand

Computer Arithmetic & Sources of Error

Floating-point representation and precision issues

Simplified

- Computers store real numbers approximately using **floating-point format**.
- Format:

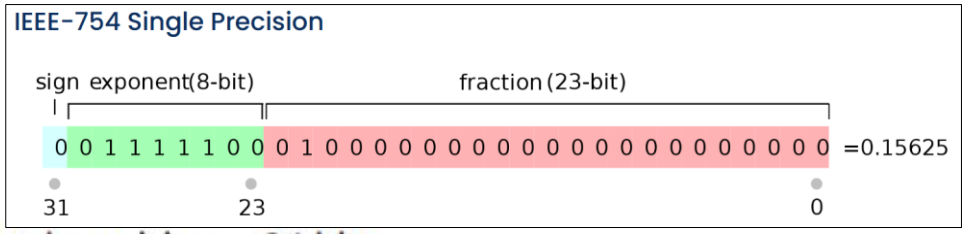
$$x = \pm(\text{significand}) \times 2^{\text{exponent}}$$

- Some decimal fractions (e.g., 0.1) cannot be exactly represented in binary—they repeat forever and must be truncated or rounded.

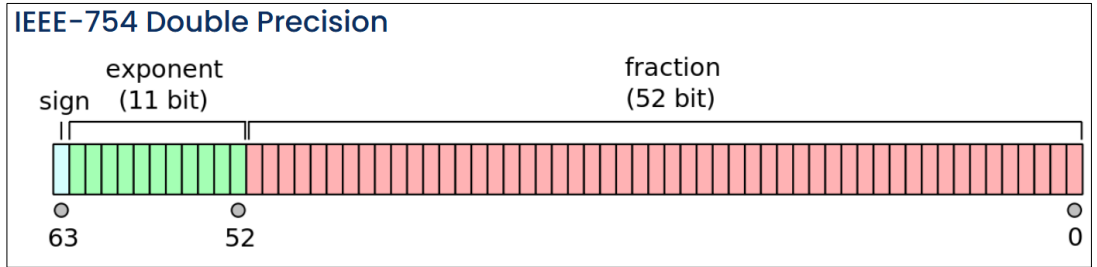
Computer Arithmetic & Sources of Error

Floating-point representation and precision issues

- **Normalization**: Numbers are adjusted so the significand starts with `1.x...`, ensuring maximum precision.
- **IEEE-754 Single Precision (32-bit)**:
  - 1 sign bit
  - 8 exponent bits
  - 23 fraction bits + an implicit leading 1 → total precision ≈ 24 bits



- **IEEE-754 Double Precision (64-bit)**:
  - 1 sign bit
  - 11 exponent bits
  - 52 fraction bits + implicit leading 1 → total precision ≈ 53 bits



Computer Arithmetic & Sources of Error

Special Values — Zero, Infinity, NaN

Zero: Represented when sign = 0 or 1, exponent = all zeros, and fraction = zero → allows +0 and –0.

Infinity (Inf): Occurs when exponent = all ones and fraction = zero → sign bit determines +∞ or –∞.

NaN ("Not a Number"): Occurs when exponent = all ones and fraction ≠ zero → represents undefined or erroneous results.

Computer Arithmetic & Sources of Error

Machine epsilon

Machine epsilon (ε_m) is the distance (gap) between 1 and the next largest floating-point number.

For IEEE-754 Single Precision:

$$\varepsilon_m = 2^{-23}$$

Shown as:

$$\varepsilon_m = 1.000000\ldots0001 - 1.000000\ldots0000 = 2^{-23}$$

(Here the significand has 23 bits: 22 stored + 1 for the last step)

For IEEE-754 Double Precision:

$$\varepsilon_m = 2^{-52}$$

Shown as:

$$\varepsilon_m = 1.000000\ldots0001 - 1.000000\ldots0000 = 2^{-52}$$

(Here the significand has 52 bits: 51 stored + 1 for the last step)

Computer Arithmetic & Sources of Error

Machine epsilon

Machine epsilon (ε_m) is the distance (gap) between 1 and the next largest floating-point number.

[ ]
✓ 0s

```python
import numpy as np

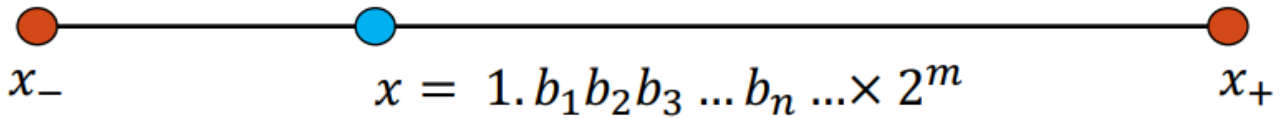
# Single Precision
eps_single = np.finfo(np.float32).eps
print(f"Single precision machine epsilon = {eps_single}")

# Double Precision
eps_double = np.finfo(np.float64).eps
print(f"Double precision machine epsilon = {eps_double}")
```

```
Single precision machine epsilon = 1.1920928955078125e-07
Double precision machine epsilon = 2.220446049250313e-16
```

Computer Arithmetic & Sources of Error

Rounding errors



$$\frac{|\tilde{x} - x|}{|x|} \leq 2^{-23} \approx 1.2 \times 10^{-7}$$

Single precision: Floating-point math consistently introduces relative errors of about 10^{-7}. Hence, single precision gives you about 7 (decimal) accurate digits.

$$\frac{|\tilde{x} - x|}{|x|} \leq 2^{-52} \approx 2.2 \times 10^{-16}$$

Double precision: Floating-point math consistently introduces relative errors of about 10^{-16}. Hence, double precision gives you about 16 (decimal) accurate digits.

Computer Arithmetic & Sources of Error

Scientific Notation, Rounding, and Truncation

Scientific Notation

In **scientific notation**, a number can be expressed in the form

$$x = \pm r \times 10^m$$

where r is a coefficient in the range $1 \leq r < 10$ and m is the exponent.

1165.7 =

0.0004728 =

Computer Arithmetic & Sources of Error

Discretization Error and Catastrophic Cancellation

Error in Numerical Methods

- Every result we compute in Numerical Methods contain errors!
- We always have them... so our job? Reduce the impact of the errors
- How can we model the error?

  Approximate result = True Value + Error

$$\bar{x} = x + \Delta x$$

- **Absolute error:** $|x - \bar{x}|$
- **Relative error:** $\dfrac{|x - \bar{x}|}{|x|}$

Computer Arithmetic & Sources of Error

Error in Numerical Methods

- Absolute errors can be misleading, depending on the magnitude of the true value x.
- For example, let's assume an absolute error $\Delta x = 0.1$

$$\boxed{x = 10^5 \quad \Rightarrow \quad \bar{x} = x + \Delta x = 10^5 + 0.1 \quad \text{(accurate result)}}$$

$$\boxed{x = 10^{-5} \quad \Rightarrow \quad \bar{x} = x + \Delta x = 10^{-5} + 0.1 \quad \text{(inaccurate result)}}$$

- Relative error is independent of magnitude.

Computer Arithmetic & Sources of Error

Significant Figures

Significant figures of a number are digits that carry meaningful information. They are digits beginning to the leftmost nonzero digit and ending with the rightmost “correct” digit, including final zeros that are exact.

Examples

Computer Arithmetic & Sources of Error

Significant Figures

Suppose x is the true value and $\hat{x}$ the approximation.

The number of significant figures tells us about how many positions of x and $\hat{x}$ agree.

Suppose the true value is

$$x = 3.141592653$$

And the approximation is

$$\hat{x} = 3.14$$

We say that $\hat{x}$ has ____ significant figures of x.

Let's try the same for:

$$2) \quad \hat{x} = 3.14159$$

$$3) \quad \hat{x} = 3.1415$$

Computer Arithmetic & Sources of Error

Significant Figures

$\hat{x}$ has ***n* significant figures** of x if $|x - \hat{x}|$ has zeros in the first n decimal places counting from the leftmost nonzero (leading) digit of x, followed by a digit from 0 to 4.

$$x = 3.141592653$$

$$\tilde{x} = 3.14159 \longrightarrow |x - \tilde{x}| = \underbrace{0.000002653}_{\text{6 zeros}} = 2.653\times10^{-6} \longrightarrow \tilde{x} \text{ has } \mathbf{6\ sf}$$

$$\tilde{x} = 3.1415 \longrightarrow |x - \tilde{x}| = \underbrace{0.000092653}_{\text{5 zeros}} = 0.92653\times10^{-4} \longrightarrow \tilde{x} \text{ has } \mathbf{4\ sf}$$

$$\tilde{x} = 3.1416 \longrightarrow |x - \tilde{x}| = \underbrace{0.000007347}_{\text{6 zeros}} = 0.7347\times10^{-5} \longrightarrow \tilde{x} \text{ has } \mathbf{5\ sf}$$

Computer Arithmetic & Sources of Error

$\hat{x}$ has ***n* significant figures** of x if $|x - \hat{x}|$ has zeros in the first n decimal places counting from the leftmost nonzero (leading) digit of x, followed by a digit from 0 to 4.

So far, we can observe that $|x - \hat{x}| \leq 5\times10^{-n}$. Note that the exact number in this example can be written in the scientific notation form $x = q\times10^{0}$. *What happens when the exponent is not zero?*

We use the relative error definition instead!

$$e_r = \frac{|x-\hat{x}|}{|x|} = \frac{|x-\hat{x}|}{q\times10^{m}} \leq \frac{5\times10^{-n}10^{m}}{q\times10^{m}} = \frac{5}{q}\times10^{-n} \leq 5\times10^{-n}$$

Computer Arithmetic & Sources of Error

Accurate to *n* significant digits means that you can trust a total of *n* digits. *Accurate digits* is a measure of relative error.

n is the number of accurate significant digits

Relative error: $error = \frac{|x_{exact} - x_{approx}|}{|x_{exact}|} \leq 5 \times 10^{-n}$

In general, we will use the rule-of-thumb:

$$\boldsymbol{error = \frac{|x_{exact} - x_{approx}|}{|x_{exact}|} \leq 10^{-n+1}}$$

For example, if relative error is 10^{-2} then $\hat{x}$ has at most 3 significant figures of x

Computer Arithmetic & Sources of Error

Sources of Error

Main source of errors in numerical computation:

- **Rounding error:** occurs when digits in a decimal point $(1/3 = 0.3333\ldots)$ are lost (0.3333) due to a limit on the memory available for storing one numerical value.
- **Truncation error:** occurs when discrete values are used to approximate a mathematical expression (e.g., the approximation $\sin(\theta) \approx \theta$ for small angles θ).

Computer Arithmetic & Sources of Error

Sequences, Convergence, and Taylor Series

Sequences and Convergence in Numerical Computation

- In numerical computation, we often generate a **sequence of approximations** and hope it converges to the true solution.
- A sequence $\{a_n\}$ is an ordered list of numbers generated by a formula or iterative procedure.
- **Convergence Definition:**

  A sequence converges to L if the terms eventually get arbitrarily close to L.
  - Formally: for any small $\varepsilon > 0$, there exists N such that

    $|a_n - L| < \varepsilon \quad \forall n > N.$
- Intuition: as $n \to \infty$, $a_n \to L$.

Computer Arithmetic & Sources of Error

Sequences, Convergence, and Taylor Series

Taylor Series Convergence

- **Taylor series** provide a powerful example of sequences & convergence.
- For e^x:

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$$

- **Partial Sum:**
  $S_n = 1 + x + x^2/2! + \cdots + x^n/n!$
  Each S_n is a polynomial approximation to e^x.
- Example for $x = 2$:
  - 0 terms → $S_0 = 1$ (very rough)
  - 1 term → $S_1 = 3$
  - 2 terms → $S_2 = 5$
  - 3 terms → $S_3 = 6.3333$
  - 9 terms → $S_9 \approx 7.388712522$ (very close to $e^2 \approx 7.389056099$)
- The **truncation error** is the difference $|e^x - S_n|$.
- Accuracy improves as n increases, provided x is within the **radius of convergence**.

Computer Arithmetic & Sources of Error

Sequences, Convergence, and Taylor Series

Convergence in Iterative Methods

- Convergence is not just for series — it is key in **iterative methods** too.
- Root-finding methods like **Newton's method** or **Bisection** generate a sequence:

$$x_0, x_1, x_2, \ldots \quad \Rightarrow \quad x_n \to \text{root}$$

- **Convergence Rate**:
  - **Linear convergence**: error decreases by roughly a constant factor each step.
  - **Quadratic convergence**: error decreases much faster; number of correct digits roughly doubles per iteration (as in Newton's method near the root).
- Understanding **whether** a sequence converges and **how fast** is essential in numerical algorithm analysis.

Computer Arithmetic & Sources of Error

Approximations using the Intermediate Value Theorem

Intermediate Value Theorem (IVT)

- **Statement of IVT:**

  If $f(x)$ is **continuous** on $[a, b]$ and
  $f(a)$ and $f(b)$ have **opposite signs**,
  then there exists $p \in (a, b)$ such that

  $$f(p) = 0$$

- **Interpretation:**

  The function **must cross the x-axis** somewhere in the interval.

- **Key Insight:**
  - IVT guarantees **existence** of at least one root.
  - But it does **not give the exact location** of the root.

Computer Arithmetic & Sources of Error

Approximations using the Intermediate Value Theorem

Bisection Method (Using IVT)

- Start with $[a, b]$ where $f(a) \cdot f(b) < 0$
- **Iteration Step:**
  1. Compute midpoint $c = (a + b)/2$
  2. Evaluate $f(c)$
  3. Choose subinterval where **sign change** occurs

     → new interval becomes $[a, c]$ or $[c, b]$
- Repeat until interval is sufficiently small.
- After n iterations:

$$\text{Interval length} = \frac{b - a}{2^n}$$

- Midpoint c gives an approximation to the root.

Computer Arithmetic & Sources of Error

Approximations using the Intermediate Value Theorem

Properties and Convergence

- The **error bound** after n steps is simply half the current interval length — so we always know how far we might be from the root.
- Example: if the initial interval is 1 unit long, after 10 iterations it becomes $1/2^{10} \approx 0.001$, so we are guaranteed that the true root is within about 0.001 of our current midpoint.
- **Convergence is linear**: the error is cut in half each step.
- While bisection is slower than other methods (like Newton's), it is **guaranteed to converge** as long as $f(x)$ is continuous and $f(a) \cdot f(b) < 0$.
- This makes it a **robust first choice** when we just need to find a root safely.

Next

- **Introduction & Python for Numerical Computing**
- **Computer Arithmetic & Sources of Error**
- **Roots of Equations**
  - Bisection method
  - Fixed-point iteration
  - Newton–Raphson method
  - Secant method
  - Implementations in Python
- Interpolation & Approximation
- Numerical Integration
- Numerical Differentiation & ODEs
- Numerical Linear Algebra
- Project